Department of
Computer Science and Engineering

Title: Thread Management

Operating System Lab
CSE 310

Green University of Bangladesh

1 Objective(s)

- To gather knowledge of threads.
- To learn about efficient multitasking and resource management

2 Threads and process

GNU/Linux presents two fundamental types of processes. These are kernel threads and user processes. Multithreaded applications are a useful paradigm for system development because they offer many facilities not available to traditional GNU/Linux processes. Both processes and threads have control flows and can run concurrently, but they differ in some very distinct ways. Threads, for example, share data, where processes explicitly don't. When a thread is created, the only new element created is a stack that is unique for the thread. The code and global data are common between the threads. The shared data aspect of threads is probably the most difficult to design around, but it is also powerful and can lead to simpler applications with higher performance. The threads API returns 0 on success but a positive value to indicate an error.

2.1 The pthreads API

All multithreaded programs must make the pthread function prototypes and symbols available for use. This is accomplished by including the pthread standard header. All multithreaded applications must create threads and ultimately destroy them. The pthread_create function permits the creation of a new thread, whereas pthread_exit allows a thread to terminate itself. To create a new thread, you call pthread_create and associate your pthread_t object with a function (start_routine).

Most languages have the concept of loops: If we want to repeat a task twenty times, we don't want to have to type in the code twenty times, with maybe a slight change each time. As a result, we have for, until and while loops in the Bourne shell. This has somewhat fewer features than other languages, but nobody claims that shell programming has the power of C.

2.1.1 Creating a Thread with pthread_create

```

#include <pthread.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>

int main()
{
int ret;

pthread_t mythread;

ret = pthread_create( &mythread, NULL, myThread, NULL );
if (ret != 0) {

    printf( " C a n t  create pthread (%s)\n", strerror(errno) );

    exit(-1);
}
return 0;

}
```

The pthread_create function returns zero if successful; otherwise, a nonzero value is returned.

 © Dept. of Computer Science and Engineering, GUB

2.1.2 Terminating a Thread with pthread_exit

```

void *myThread( void *arg )
{
    printf (*Thread ran! \n") ;

    /* Terminate the thread */
    pthread_exit( NULL );
}
```

The exit value presented to pthread_exit must not be of local scope; otherwise, it won't exist after the thread is destroyed. The pthread_exit function does not return.

2.2 THREAD MANAGEMENT

Before digging into thread synchronization and coordination, it's time to look at a couple of miscellaneous thread functions that can be of use.

2.2.1 Retrieving the pthread_t Handle with pthread_self

The first is the pthread_self function. It can be used by a thread to retrieve its unique identifier. The thread itself can also retrieve this identifier by calling pthread_self.

```
void *myThread( void *arg )
{

pthread_t pt;

pt = pthread_self();

printf("Thread % ran\n", (int)pt );

pthread_exit( NULL );

}
```

2.2.2 Providing a Single-Use Initialization Function with pthread_once

Most applications require some type of initialization, but with threaded applications, the job can be difficult. The pthread_once function allows a developer to create an initialization routine that is invoked for a multithreaded application only once (even though multiple threads might attempt to invoke it).

```
#include <pthread.h>
pthread_once_t my_init_mutex = pthread_once_init;
void initialize_app( void )

{

/* Single-time init here */

}

void *myThread( void *arg )
{
....
pthread_once( &my _init_mutex, initialize_app );
....
}
```

 © Dept. of Computer Science and Engineering, GUB

2.3 THREAD SYNCHRONIZATION

The ability to synchronize threads is an important aspect of multithreaded application development. The most basic method is the ability for the creator thread to wait for the created thread to finish (otherwise known as a join). This activity is provided by the pthread_join API function

2.3.1 Joining Threads with pthread_join

When called, the pthread_join call suspends the calling thread until a join is complete. When the join is done, the caller receives the joined thread's termination status as the return from pthread_join. A thread is automatically joinable when using the default attributes of pthread_create. If the attribute for the thread is defined as detached, then the thread can't be joined (because it's detached from the creating thread).

```
#include <pthread.h>
#include <stdlib.h>
#include <stdio.h>
void *myThread(void *arg)
{
    printf("Thread %d started\n", (int) arg );
    pthread_exit(arg);
}
#define MAX_THREADS 5
int main()
{
int ret, i, status;
pthread_t threadIds[MAX_THREADS];
for (i = 0; i < MAX_THREADS ; i++) {
    ret = pthread_create( &threadIds[i], NULL, myThread, (void *)i );
    if (ret != 0) {
    printf("Error creating thread %d\n", (int) threadIds[i]);
    }
  }
}
for (i = 0; i < MAX_THREADS ; i++) {
ret = pthread_join( threadIds[i], (void **)&status );
if (ret != 0) {
    printf("Error joining thread %d\n", (int) threadIds[i]);
}
else {
    printf("Status = %d\n", status );
}
}
return 0;
}
```

2.3.2 Detaching a Thread from Within with pthread_detach

When this thread exits, all resources are immediately freed (as it's detached and will never be joined by another thread). The pthread_detach function returns zero on success, nonzero if an error occurs. GNU/Linux automatically places a newly created thread into the joinable state. This is not the case in other implementations, which can default to detached.

```
void *myThread( void *arg )
{
printf( "Thread %d started\n", (int)arg );

pthread_detach( pthread_self() );

pthread_exit( arg );
}
```

 © Dept. of Computer Science and Engineering, GUB

3 Lab Task (Please implement yourself and show the output to the instructor)

- Implement Dining Philosophers Problem using threads

4 Lab Exercise (Submit as a report)

- Implement producer-consumer problem using threads.

5 Discussion Conclusion

Based on the focused objective(s) to understand thread management, the additional lab exercise made me more confident in the fulfillment of the objectives(s).

6 Policy

Copying from the internet, classmates, seniors, or from any other source is strongly prohibited. 100% marks will be deducted if any such copying is detected

 © Dept. of Computer Science and Engineering, GUB